МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №1

з дисципліни «Алгоритми та структури даних»

на тему *«Рекурсивні алгоритми»*

Виконав:
студент групи ІМ-о51
Ушканов Т.А.

Перевірив:
ас. Череватенко О.В.

Київ 2026

Завдання

Варіант № 18

$$F_1 = 1; \quad F_{i+1} = F_i \cdot x^2/(4i^2 - 2i), \quad i > 0;$$
$$\sum_{i=1}^{n} F_i = \operatorname{ch} x, \quad |x| < 10^6.$$

Постановка задачі:

Дане натуральне число n. Знайти суму перших n членів ряду чисел, заданого рекурентною формулою. Розв'язати задачу трьома способами:

1) у програмі використати рекурсивну функцію, яка виконує обчислення і членів ряду, і суми на рекурсивному спуску;

2) у програмі використати рекурсивну функцію, яка виконує обчислення і членів ряду, і суми на рекурсивному поверненні;

3) у програмі використати рекурсивну функцію, яка виконує обчислення членів ряду на рекурсивному спуску, а обчислення суми на рекурсивному поверненні.

Тестування програм

1. З метою тестування потрібно написати циклічний варіант рішення задачі, а також перевірити правильність обчислень елементів ряду та їх суми за допомогою калькулятора.

2. Як результат, роздрукувати дані тестування усіма трьома рекурсивними функціями та циклічною програмою, а також навести скриншот обчислень на калькуляторі. Для тестування прийняти n = 5, а значення x вибрати самостійно (у межах області визначення функції). Результати обчислень усіма способами повинні співпадати.

Текст програм:

1.

```
public class RecursionOne
{
    private static double F(int i, int n, double x, ref double sum)
    {
        double fi;

        if (i == 1)
        {
            fi = 1;
        }
        else
        {
            fi = F(i - 1, n, x, ref sum) * (x * x) / (4.0 * (i - 1) * (i - 1) - 2.0 * (i -
1));
        }

        Console.WriteLine($"F({i}) result = {fi}");

        sum += fi;
        return fi;
    }

    public static double Compute(int n, double x)
    {
        if (n <= 0)
            throw new ArgumentOutOfRangeException(nameof(n), "Value of n must be
positive");
        if (Math.Abs(x) >= 1_000_000)
            throw new ArgumentOutOfRangeException(nameof(x), "x must satisfy |x| < 10^6");

        Console.WriteLine($"n = {n}; x = {x}");

        double sum = 0;
        F(n, n, x, ref sum);
        Console.WriteLine($"sum = {sum}");
        return sum;
    }
}
```

2.

```
public class RecursionTwo
{
    private static (double fi, double sum) F(int i, double x)
    {
        if (i == 1)
        {
            Console.WriteLine($"F({i}) result = {1}");
            return (1.0, 1.0);
        }
```

```
        var (prevFi, prevSum) = F(i - 1, x);
        double fi = prevFi * (x * x) / (4.0 * (i - 1) * (i - 1) - 2.0 * (i - 1));
        Console.WriteLine($"F({i}) result = {fi}");
        return (fi, prevSum + fi);
    }

    public static double Compute(int n, double x)
    {
        if (n <= 0)
            throw new ArgumentOutOfRangeException(nameof(n), "Value of n must be
positive");
        if (Math.Abs(x) >= 1_000_000)
            throw new ArgumentOutOfRangeException(nameof(x), "x must satisfy |x| < 10^6");

        Console.WriteLine($"n = {n}; x = {x}");

        var (_, sum) = F(n, x);
        Console.WriteLine($"sum = {sum}");
        return sum;
    }
}
```

3.

```
public class RecursionThree
{
    private static double F(int i, double x, out double sum)
    {
        if (i == 1)
        {
            Console.WriteLine($"F({i}) result = {1}");
            sum = 1.0;
            return 1.0;
        }

        double prevFi = F(i - 1, x, out double prevSum);
        double fi = prevFi * (x * x) / (4.0 * (i - 1) * (i - 1) - 2.0 * (i - 1));
        sum = prevSum + fi;
        Console.WriteLine($"F({i}) result = {fi}");
        return fi;
    }

    public static double Compute(int n, double x)
    {
        if (n <= 0)
            throw new ArgumentOutOfRangeException(nameof(n), "Value of n must be
positive");
        if (Math.Abs(x) >= 1_000_000)
            throw new ArgumentOutOfRangeException(nameof(x), "x must satisfy |x| < 10^6");

        Console.WriteLine($"n = {n}; x = {x}");

        F(n, x, out double sum);
```

```
        Console.WriteLine($"sum = {sum}");
        return sum;
    }
}
```

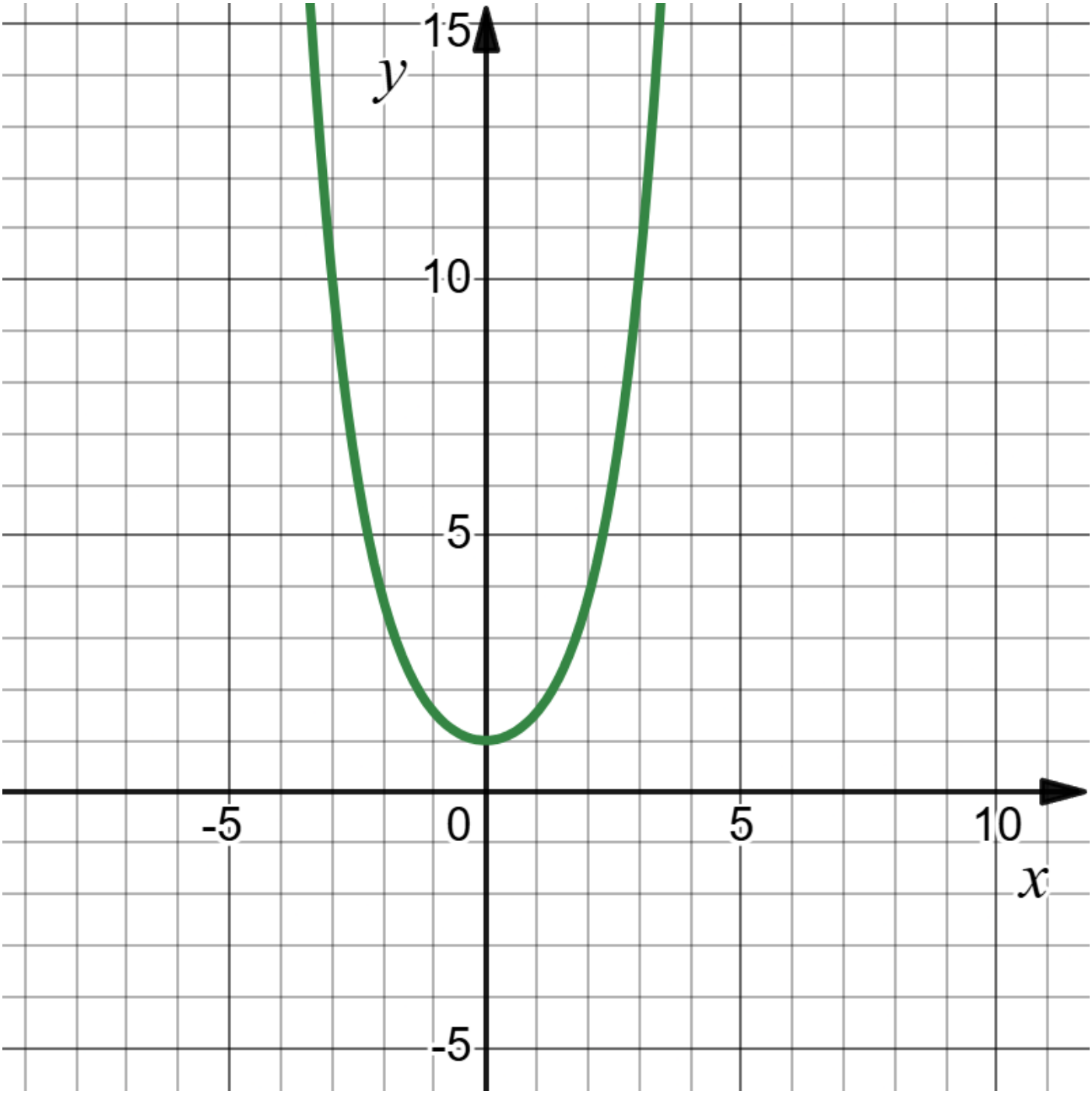
Тестування:

```
Програма 1:
n = 5; x = 10
F(1) result = 1
F(2) result = 50
F(3) result = 416,6666666666667
F(4) result = 1388,8888888888891
F(5) result = 2480,1587301587306
sum = 4336,714285714286
```

```
Програма 2:
n = 5; x = 5
F(1) result = 1
F(2) result = 12,5
F(3) result = 26,041666666666668
F(4) result = 21,701388888888893
F(5) result = 9,688120039682541
sum = 70,9311755952381
```

```
Програма 3:
n = 5; x = 4,670000076293945
F(1) result = 1
F(2) result = 10,904450356292728
F(3) result = 19,81783959547543
F(4) result = 14,406843202522278
F(5) result = 5,610668089028482
sum = 51,739801243318915
```

Графік:



Висновок:

У лабораторній роботі №1 була опрацьована рекурсивних функцій. Було реалізовано три варіанти рекурсивної функції для обчислення членів ряду, і суми. На основі даних виведених з цієї функції було побудовано Графік.